

Vision-Based Line Following for a Mobile Robot

Sean Biscoe-F325508

Abstract—We present a vision-based line-following system for a mobile robot using an ImageNet-pretrained ResNet-18 on a Jetson Orin Nano. The task is formulated as 5-class steering classification, with predictions filtered through a confidence-gated majority-vote buffer and mapped to per-wheel motor commands. We trained three model variants on a manually labelled 5,632-frame dataset and used systematic error analysis to correct 269 mislabelled frames, increasing test accuracy from 71.2% to 80.5%. Per-class recall revealed a clear imbalance between left- and right-turn classes (89%/52% vs. 80%/42%), reflecting the skew in the training data. Deployment testing on a closed-loop track showed that test accuracy was a poor predictor of real-world performance: the highest-accuracy variant (80.5%) systematically failed at right-turning corners, while a lower-accuracy variant (77.7%) with better recall on the rare `right` class proved more reliable in practice. Per-corner analysis confirmed that the failure mode was class-specific rather than direction-specific. These results highlight the importance of class balance, hardware asymmetry, and camera alignment when evaluating learned controllers for asymmetric control tasks.

Index Terms—mobile robot, line following, deep learning, ResNet-18, visual control

I. INTRODUCTION

Autonomous line following is a core problem in mobile robotics, with uses in warehouse logistics, factory automation, and educational robot platforms. Although the idea is simple enough, keep the robot on a marked path, making it work reliably in real time is much harder. The system has to produce stable steering commands at camera speed, cope with changes in lighting and surface appearance, and recover cleanly from small steering errors without losing the line.

Traditional line-following systems often use hand-engineered colour thresholds and geometric rules with a proportional controller, as in Ismail et al. [1]. These methods are lightweight and easy to interpret, but often need careful retuning for each new environment. More recently, end-to-end learned steering has become popular, where a CNN maps raw camera input directly to steering commands [2], but this comes at the cost of interpretability. For embedded platforms with limited motor interfaces, such as JetBot [3], a useful compromise is to discretise steering into a small number of classes and treat the task as multi-class classification, which is easier to label and deploy.

We use this discrete-class approach on a mobile robot setup built around a ZED2i stereo camera, a Jetson Orin Nano, and a Yukon serial motor controller that exposes named direction commands. The track is a closed loop of yellow tape on a dark floor, with sharp corners, dense tape junctions that create visual discontinuities, and an uneven mix of left- and right-turning sections. We trained a ResNet-18 classifier [4] on manually labelled video frames collected in both directions,

and deployed it with confidence-gated temporal smoothing and a differential-drive control policy within the limits of the platform’s motor API.

A. Problem Formulation

We formulate a vision-based line following as a per-frame classification task. Given an RGB image $x \in \mathbb{R}^{H \times W \times 3}$ from the forward-facing camera, the system must produce a steering class $\hat{y} \in \mathcal{C}$, where $\mathcal{C} = \{\text{hard_left}, \text{left}, \text{straight}, \text{right}, \text{hard_right}\}$, which is then mapped to a discrete motor command. The control objective is to maintain the track between the robot’s front wheels while completing a full track loop in minimum time and without leaving the track.

B. Contributions

This paper makes four contributions:

- We build a 5,632-frame dataset from our own track and show that the weak performance of the initial model (24% recall on the rare `right` class) was mainly caused by labeller bias rather than model capacity. Correcting 269 mislabelled frames increased test accuracy from 71.2% to 80.5% (Section III-E).
- We compare three model variants with different class-rebalancing strategies and show that test accuracy is a poor predictor of deployment performance. The highest-accuracy model (v3, 80.5%) failed at right-turning corners, while v2 (77.7%) proved more reliable because of its better recall on the rare `right` class (Section V).
- We design a confidence-gated majority-vote filter that drops borderline turn predictions, and measure the hardware control-rate limit of the Yukon per-wheel motor API (19 Hz, 52.6 ms per command, Section IV-D).
- We run a per-corner deployment analysis in both directions and show that the failure pattern is class-specific rather than direction-specific: right-turn corners fail wherever they appear, and these deployed failures map directly to the model’s per-class test-set recall (Section V-D).

II. RELATED WORK

A. Classical Vision-Based Line Following

Traditional line-following systems rely on hand-crafted image processing rather than learned features. Ismail et al. [1] describe a mobile robot that follows a white line on a dark surface by converting each frame to greyscale, equalising the histogram, and applying a heuristic threshold to isolate the line. The thresholded image is split into a coarse block grid, and the resulting pattern drives a proportional motor controller. The approach is cheap and interpretable, but it needs careful

per-environment tuning and produces noticeably jerky motion. For our track, which has a yellow line broken by gaps at every tape junction, this kind of approach would mean retuning both the colour threshold and the block-grid logic for every lighting condition, and would still struggle at the discontinuities.

B. End-to-End Learned Steering

A more recent alternative is end-to-end learning, where a CNN maps raw camera input straight to steering commands and skips feature engineering entirely. Bojarski et al. [2] showed that such a model can learn road-relevant features from human driving data and predict steering from a single camera image. The trade-off is that these models typically need much more data than is practical for a coursework-scale setup, and their failures are opaque: when the network gets it wrong there is no intermediate signal to debug, which makes targeted dataset improvements harder. We avoid this by using transfer learning, a discrete steering output, and the iterative dataset refinement described in Section III-E.

C. Discrete-Class Steering for Embedded Systems

For resource-constrained platforms such as the JetBot reference design [3], a practical approach is to discretise steering into a small set of classes, typically three to five, and formulate the task as multi-class classification. This reduces annotation complexity, simplifies deployment on embedded GPUs, and integrates well with discrete motor control interfaces, such as the named-direction commands used in our setup. Our approach adopts this paradigm using an ImageNet-pretrained ResNet-18 [4] as a backbone, together with temporal smoothing and explicit handling of class imbalance caused by track geometry.

D. Temporal Aggregation in Video Classification

Per-frame CNN predictions over a video stream are inherently noisy, making some form of temporal aggregation desirable for stable downstream control. Karpathy et al. [5] evaluate several temporal fusion strategies for video CNNs on a large-scale dataset and show that aggregating predictions over short clips improves performance over feature-based baselines, while also indicating that relatively lightweight aggregation can be effective in practice. Standard approaches include majority voting, score averaging, and learned recurrent aggregation. We use a variant of majority voting: a confidence-gated 3-frame majority-vote filter that pre-rejects low-confidence non-straight predictions before aggregation, trading a small amount of latency for stability against single-frame errors.

III. DATASETS

A. Data Collection

To build our dataset, we manually steered the robot along the track for 12 video laps, six videos in each direction. We captured the footage using the ZED2i's left camera at VGA resolution (672×376) and saved it as MP4s at the camera's native frame rate. To filter out redundant, nearly identical footage, we sampled every fifth frame, resulting in a final set of 5,632 RGB images.

B. Preprocessing

We cropped the top 200 pixels from each frame to strip away the ceiling and background, focusing the model's attention entirely on the floor and the track line. To prepare these images for a ResNet-18 backbone, we resized them to a 256-pixel short side and took a 224×224 centre crop. Finally, we normalised the pixel values using the standard ImageNet mean and standard deviation.

C. Labelling

Each frame was hand-labelled into one of five steering classes: `hard_left`, `left`, `straight`, `right`, and `hard_right`. We chose a discrete classification approach over continuous regression for three main reasons. First, the robot's serial interface only accepts specific direction commands, making discrete labels a natural fit for the hardware. Second, human labellers can categorise frames more quickly and consistently than they can estimate precise angles. Finally, a discrete label stream is easier to smooth over time, allowing us to filter out single-frame errors more effectively than we could with regression.

D. Train/Validation/Test Split

We split the dataset at the video level rather than by frame, with four videos for training, one for validation, and one for testing per direction ($\sim 70/15/15$). A frame-level random split would inflate test accuracy because adjacent frames are nearly identical, so the video-level split forces the model to generalise to unseen recording sessions, at the cost of higher-variance test estimates.

E. Label Refinement

Our initial training of the ResNet-18 classifier yielded a 71.2% test accuracy, but we noticed a significant performance gap between classes. Specifically, the model only correctly identified right turns 24% of the time, compared to an 80% success rate for `hard_right`. An error analysis showed that 81% of these `right-as-straight` confusions occurred within a single test video. When we visually inspected these frames (see Figure 1), we discovered a "labeller bias": the frames had originally been labelled according to the operator's intended turn at a corner, even though the line still appeared straight in the current view. To fix this, we performed a manual relabelling pass focused on all `right` and `left` frames, ensuring each tag matched the visual content rather than the driver's intent. 269 frames (4.8% of the dataset) were corrected: 197 in the `right` class (predominantly to `straight`) and 72 in the `left` class. The final class distribution is shown in Table I.

The 24:1 ratio between the most and least represented classes is addressed during training via a `WeightedRandomSampler` and class-weighted cross-entropy loss (Section IV-C).



Fig. 1: Examples of frames originally labelled as *right* but predicted as *straight* by the v1 classifier. Visual inspection confirms the line appears predominantly vertical, indicating the labels reflected the operator’s anticipated next action rather than the current visual content. These and similar frames were corrected during the manual relabelling pass described below.

TABLE I: Class distribution before and after the manual relabelling pass.

Class	Original	After relabelling
straight	3,850	4,067
hard_left	549	573
hard_right	482	509
right	511	315
left	240	168
Total	5,632	5,632

IV. METHOD

A. System Overview

The whole system runs onboard the Jetson Orin Nano. As shown in Figure 2, each frame from the ZED2i camera is preprocessed and passed through a ResNet-18 classifier that predicts one of five steering classes. The prediction is then smoothed by a temporal filter before being mapped to a motor command and sent to the Yukon serial controller.

B. Network Architecture

We use a ResNet-18 [4] pretrained on ImageNet [6] and fine-tuned on our 5,632-frame dataset, with the original 1,000-class output layer replaced by a randomly initialised linear layer producing five logits for the steering classes from Section III-C. ResNet-18 fits this task well: the residual connections keep training stable on a small dataset, the ImageNet features transfer cleanly to picking out a thin yellow line on a dark floor, and at 11M parameters the network runs in real time on the Orin Nano without bottlenecking camera capture.

We considered two alternatives. A steering-angle regression head can produce smoother outputs [2], but it is sensitive to label noise and needs careful loss-shaping to train stably, both of which are problematic for our small, manually-labelled dataset. A classical HSV-threshold plus PID baseline would be lightweight, but reliably segmenting a yellow line broken by dark tape gaps would mean retuning colour thresholds for every lighting condition, removing the generalisation benefit of a learned model. Discrete classification with temporal smoothing sits between the two, offering stable training, easy deployment, and resilience to the discontinuities that would break a threshold-based pipeline.

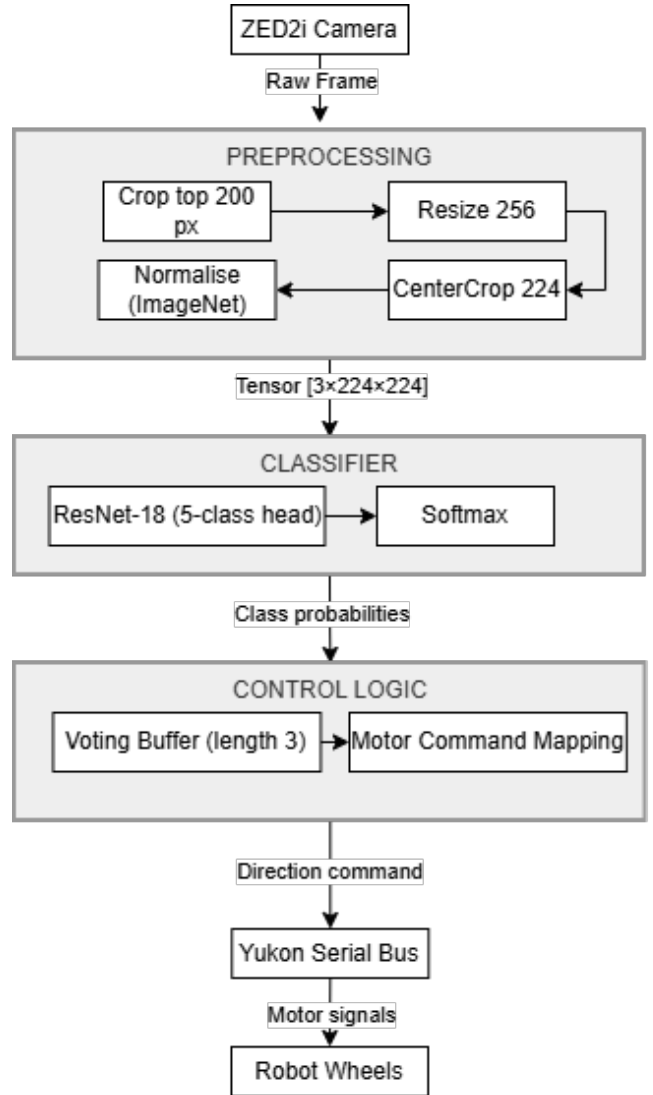


Fig. 2: Overall system architecture. The pipeline runs onboard the Jetson Orin Nano at ~ 20 Hz: each ZED2i frame is cropped, normalised, classified by ResNet-18, smoothed by a length-3 temporal filter, and mapped to a Yukon serial command.

C. Training

We trained the model using weighted cross-entropy loss and a `WeightedRandomSampler` to handle our imbalanced dataset on the training split (Section III-D). The classifier’s output logits $z \in \mathbb{R}^K$ are converted to class probabilities via the softmax function:

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}, \quad i = 1, \dots, K \quad (1)$$

where $K = 5$ is the number of steering classes (Eq. 1). The model is trained with class-weighted cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^K w_i y_i \log p_i \quad (2)$$

where $y_i \in \{0, 1\}$ is the one-hot ground-truth label and w_i is the weight assigned to class i (Eq. 2). Both the loss weights

and the `WeightedRandomSampler` weights are derived from inverse class frequency (Eq. 3):

$$w_c = \frac{N}{K \cdot n_c} \quad (3)$$

where N is the total number of training frames and n_c is the count of class c . To prevent the model from overfitting to the 168 `left` frames under aggressive oversampling, the sampler weights were softened to $w_c^{0.3}$, while the loss weights retained the full inverse-frequency form.

We optimised with Adam [7] at learning rate 1×10^{-4} , halved every five epochs, for 20 epochs at batch size 32, selecting the best checkpoint by validation accuracy.

Three augmentations improve deployment generalisation: *ColorJitter* (lighting variation), small affine transformations (5% translation and scaling, for camera shifts), and *RandomErasing* ($p = 0.3$, for tape gaps).

D. Inference and Temporal Smoothing

The system runs at approximately 20 Hz on the Jetson Orin Nano. The vision model processes frames at ~ 30 ms, but control is limited by the motor interface, which we measured at 52.6 ms per call (19 Hz) over 50 sequential `set_wheels` calls, the primary bottleneck.

To reduce prediction jitter during deployment, two lightweight filtering steps were applied. First, a confidence gate suppresses low-confidence turn predictions: if a non-straight class is predicted with $\max_i p_i < 0.55$, it is re-labelled as `straight`. This prevents uncertain predictions from triggering premature turns on straight track segments; Equation 4 formalises the gate:

$$\tilde{p} = \begin{cases} \text{straight} & \text{if } \arg\max_i p_i \neq \text{straight} \\ & \text{and } \max_i p_i < 0.55 \\ \arg\max_i p_i & \text{otherwise} \end{cases} \quad (4)$$

Second, predictions are smoothed using a 3-frame majority-vote buffer. This removes isolated misclassifications while keeping the response delay low (~ 150 ms); longer 5-frame windows introduced visible delay during early testing.

E. Control Strategy

Because the Yukon serial interface only exposes named direction commands and individual wheel speeds, rather than a single continuous arc-steering command, we compared two control strategies.

Spin-burst (S1): A "spin-and-nudge" sequence prevents the robot getting stuck at sharp corners. Each non-straight prediction triggers a brief `spinLeft` or `spinRight` command (faster for hard turns, slower for gentle ones), followed by a short forward nudge to keep forward progress through the corner.

Differential-drive arcs (S2): We implemented a per-wheel variant using `set_wheels(left, right)` commands with asymmetric speeds: equal for `straight`, moderate differential for `left/right`, and aggressive differential

Algorithm 1 Per-frame control loop with confidence-gated majority vote

```

1: buffer  $\leftarrow$  empty deque (max length 3)
2: loop
3:   frame  $\leftarrow$  camera.read()
4:   cropped  $\leftarrow$  frame.crop(top=200)
5:   x  $\leftarrow$  normalise(centre_crop(resize(cropped)))
6:   logits  $\leftarrow$  model(x)
7:   probs  $\leftarrow$  softmax(logits)
8:   pred  $\leftarrow$  argmax(probs)
9:   conf  $\leftarrow$  max(probs)
10:  if pred  $\neq$  straight and conf  $<$  0.55 then
11:    pred  $\leftarrow$  straight
12:  end if
13:  buffer.append(pred)
14:  final  $\leftarrow$  argmaxc  $|\{p \in \textit{buffer} : p = c\}| \triangleright$  majority vote
15:  execute_command(final)
16: end loop

```

approaching pure rotation for `hard_left/hard_right`. The Yukon controller's per-wheel API is hardware-limited to 19 Hz (Section IV-D), but this remains within the bandwidth of our temporal filter. We selected S2 for deployment because it produced visibly smoother trajectories than S1; the quantitative comparison is reported in Section V-B.

Both strategies share the same vision pipeline and confidence-gated majority-vote filter (Section IV-D); they differ only in how the predicted class is assigned to motor activity. Comparison of their lap times and stability is reported in Section V.

V. EXPERIMENTS

A. Model Variant Comparison

We trained three model variants to isolate the effects of data quality and sampling strategy (Table II). The v1 baseline trained on the original labels reached 71.2% test accuracy, but with only 24% recall on the `right` class. Manual error analysis (Section III-E) traced this to systematic mislabelling of corner-approach frames. Retraining on the corrected labels (v2) raised overall accuracy to 77.7% and `right` recall to 42%, although `straight` recall fell from 85% to 77% as aggressive oversampling pulled the model away from the dominant class. The v3 variant uses a softer $w^{0.3}$ inverse-frequency weighting, recovering `straight` recall to 86% and reaching the best overall accuracy of 80.5%, but at the cost of some rare-class recall.

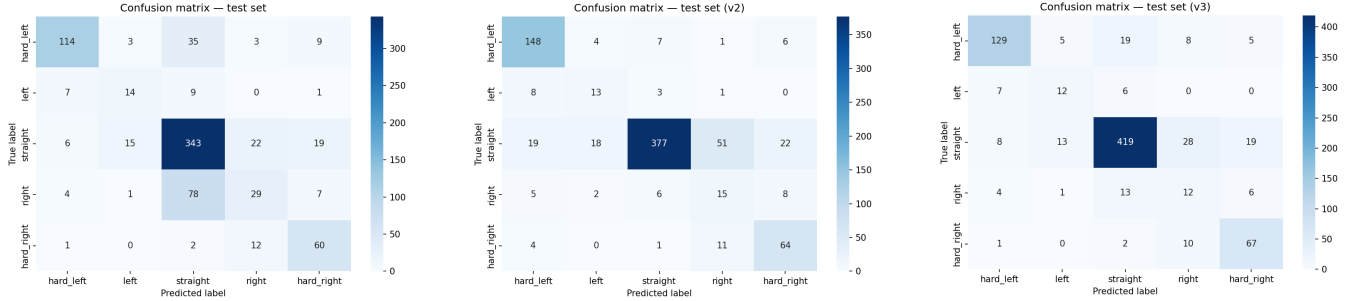
We initially picked v3 for its higher accuracy, but it failed at right-turning corners on the track, where its 33% `right` recall left almost no margin. v2's 42% `right` recall was more reliable in practice, so v2 was used for all deployment experiments (Sections V-B to V-E).

B. Control Strategy Comparison

We compared S1 (spin-burst) and S2 (differential-drive arcs) from Section IV-E under identical vision-pipeline settings (v2 model, confidence 0.55, 3-frame majority vote). S2 produced visibly smoother and faster trajectories on straight sections and

TABLE II: Comparison of model variants on the held-out test set. v1 uses original labels with square-root softened sampling; v2 corrects labelling errors and uses full inverse-frequency sampling; v3 retains corrected labels but applies $w^{0.3}$ softened sampling. Bold marks the best per-row.

Model	Test acc (%)	Per-class recall (%)					Notes
		hard_left	left	straight	right	hard_right	
v1	71.2	70	45	85	24	80	Baseline, noisy labels
v2	77.7	89	52	77	42	80	Relabelled, $w^{1.0}$ sampling
v3	80.5	78	48	86	33	84	Relabelled, $w^{0.3}$ sampling



(a) v1: 71.2% test accuracy. Original labels. (b) v2: 77.7%. Relabelled, full reweighting. (c) v3: 80.5%. Relabelled, $w^{0.3}$ sampling.

Fig. 3: Confusion matrices on the held-out test set across the three model variants. Increasing diagonal intensity from (a) to (c) reflects improved classifier accuracy after label refinement and tuned class sampling. Note in particular the reduction of off-diagonal mass in the *right* row from (a) to (b), confirming the effect of the relabelling pass.

corner approaches, thanks to its continuous wheel-speed differentials rather than S1’s discrete spin-then-nudge sequence. The reliability difference was clearest on sharp corners where S1’s spin-then-nudge introduced visible hesitation. The 19 Hz cap on the per-wheel API (Section IV-D) is slightly below the vision pipeline’s 20 Hz, but the temporal filter absorbs the mismatch. Lab time did not allow a quantitative S1 vs S2 lap-time comparison, so we selected S2 based on qualitative trajectory smoothness and reliability of corner completion under matched parameters.

C. Lap Time and Stability

We timed the deployed system (v2, S2, confidence 0.55, 3-frame majority vote) in both track directions. The robot started from a marked position, with runtime measured from motion onset to an off-track event or lap completion. Five runs per direction are summarised in Table III.

TABLE III: Time-to-failure statistics for the deployed system across both track directions ($n = 5$ per direction). All runs in both directions failed before completing a full autonomous lap; manual intervention enabled completion of a full lap in 2:15 (Direction A, 3 interventions) and 2:26 (Direction B, 4 interventions).

Direction	Mean (s)	Std. dev. (s)	Failure point
A	59.8	7.2	Post-corner straight (corners 3–4)
B	35.8	1.3	3rd corner (90°)

Direction A survived roughly 67% longer than Direction B before going off-track. A Welch’s two-sample t -test on the run

times (Direction A: 59.8 ± 7.2 s; Direction B: 35.8 ± 1.3 s) gives $t = 7.4$ ($df = 4.3$), $p < 0.01$, so the asymmetry is statistically significant despite the small samples. All five Direction B runs failed at the same corner with very low variance ($\sigma = 1.3$ s), pointing to a reproducible failure mode rather than stochastic behaviour. Direction A failures clustered on post-corner straights, where the model struggled to re-acquire the line centre after large rotations.

A manual-assisted lap completed in 2:15 for Direction A (3 interventions) and 2:26 for Direction B (4 interventions), with the robot lifted back on track at each off-track event. The system therefore handles most of the track autonomously, with failures concentrated at specific geometric features rather than distributed uniformly.

Although consecutive autonomous laps were not achieved, the per-corner success rates in Section V-D show every corner except right-turning ones completing reliably under repeated trials. The missing full-lap capability is bottlenecked by a single class-recall deficit rather than general instability.

D. Direction Symmetry

To find which corners failed most, we ran each corner repeatedly under identical deployment parameters and recorded success rates (Table IV).

The same physical corners produce very different outcomes depending on turn direction. The sharpest corner of the track completes 4/5 as a left-turn in Direction A but 0/5 as a right-turn in Direction B. Even within Direction A, the right turn immediately after the sharpest corner failed on every attempt; the robot completes the left-turn cleanly but then cannot execute the right-turn that follows. This pattern mirrors

TABLE IV: Per-corner success rates in both directions (autonomous completion of the corner itself, without intervention) under fixed deployment parameters. Sample sizes vary by corner due to lab-time constraints. The same physical corners produce starkly different outcomes in opposite directions, where the corner is encountered as a left-turn vs. a right-turn.

Corner type	Approx. angle	Direction A	Direction B
Sharpest corner	–	4/5	0/5
90° left turn	90°	5/5	–
Right turn after sharpest corner	–	0/5	–
90° right turn (a)	90°	–	1/10
90° right turn (b)	90°	–	3/10
110° corner	110°	–	0/10
Triangle corner	~135°	–	0/10

the per-class recall from the test set (Table II): v2 achieves 89% recall on `hard_left` and 52% on `left`, but only 80% on `hard_right` and 42% on `right`. The failure mode is therefore turn-class-specific rather than direction-specific: the model fails on right-turning corners wherever they appear.

We see three factors driving this asymmetry. First, the model’s lower recall on right-turn classes reflects under-representation of right-turn frames in the training set after relabelling (Section III-E). Second, the robot showed visibly reduced traction on one wheel during testing. Third, the camera mount appeared slightly off-axis during deployment, which disproportionately affected right-turn approach frames. Each is a clear target for future iterations.

A second failure mode appears in Direction A. The corners themselves complete reliably (Table IV), but the robot frequently fails on the straight sections immediately after corner exits, going off-track on 4/5 attempts at one 90° corner and 3/5 at another. We trace this to dense tape segments that resemble line discontinuities the model was trained to absorb but appear at a frequency the training set under-represented. The pattern is consistent: drift off-centre 3–5 seconds after exiting the corner, on the densest tape sections.

E. Robustness Observations

A formal robustness experiment was not conducted within the lab session. The deployed system did show several robustness-relevant behaviours during normal runs, which we report as observational findings. The RandomErasing augmentation (Section IV-C) successfully handled isolated tape discontinuities, although the densest sequential tape sections still caused off-centre drift and spurious turns, which were the dominant Direction A failure mode (Section V-C). After each off-track event, manually replacing the robot was sufficient for autonomous operation to resume without restart, suggesting no hidden state depends on continuous tracking. Lighting varied subtly across the track due to overhead fixtures and adjacent windows but produced no observed failures, suggesting the ColorJitter augmentation was sufficient for this environment.

VI. CONCLUSION

We built a vision-based line-following system using an ImageNet-pretrained ResNet-18 with confidence-gated tempo-

ral smoothing and per-wheel differential drive on a Jetson Orin Nano. Three model variants were compared under different class-rebalancing strategies, and we deployed v2 because its higher recall on the rare `right` class proved more predictive of lap-time performance than overall test accuracy. Our main finding is that headline test accuracy is a poor predictor of deployed performance for this kind of asymmetric task. Although v3 achieved 80.5% test accuracy compared with v2’s 77.7%, it failed at right-turning corners because of its weaker recall on the rare right-turn classes. Right-turning corners failed wherever they appeared, including a right-turn within Direction A that failed on every attempt despite Direction A’s longer time-to-failure (Section V-D).

We attribute the deployed asymmetry to three factors: training-data imbalance favouring left turns, reduced traction on one wheel, and a slightly off-axis camera mount. A second failure mode appeared on dense-tape sections in Direction A, where the robot drifted off-centre on post-corner straights despite the RandomErasing augmentation intended to help with such cases.

A few limitations remain. Our robustness analysis (Section V-E) is observational rather than experimental, and a fuller study would require controlled occlusion, lighting variation, and different starting conditions. The per-corner experiments were also limited by lab time ($n = 5\text{--}10$ per corner), which weakens the statistical strength of the direction-asymmetry claim. The hardware issues were identified but not quantified. The S1 vs S2 comparison was qualitative: S2 was clearly smoother and more reliable, but we did not have robot time to gather lap-time numbers.

Future work should focus on collecting more right-turn training data, correcting the camera mount, fixing the traction asymmetry, and adding denser tape-sequence imagery to improve post-corner recovery. A promising extension would be a confidence-aware recovery state that slows down or searches for the line when predictions are uncertain.

REFERENCES

- [1] A. Ismail, H. Ramli, M. Ahmad, and M. Marhaban, “Vision-based system for line following mobile robot,” in *Proceedings of the 2009 IEEE Symposium on Industrial Electronics and Applications (ISIEA)*, vol. 2, pp. 642–645, 2009.
- [2] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, “End to end learning for self-driving cars,” 2016.
- [3] NVIDIA, “JetBot: An open-source robot based on NVIDIA Jetson Nano.” <https://github.com/NVIDIA-AI-IOT/jetbot>, 2023. Accessed: 2026-05-03.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [5] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei, “Large-scale video classification with convolutional neural networks,” in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1725–1732, 2014.
- [6] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “Imagenet large scale visual recognition challenge,” *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.
- [7] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017.